Layerflow vs DI-01 Debug Dojo: readiness check

An internal team note that compares Layerflow with the organisers' DI-01 statement. Checked against the code and the latest harness run (`harness/results/latest.json`, 17 Oct 2026, 01:25 IST).

> **Verdict:** every mandatory requirement is built and matches the statement.
> Three things still need fixing before the freeze:
>
> 1. **The sandbox red-team test is at 14/15.** The disk-flood program is labelled `timeout` instead of `runtime_error`.
> 2. **Rewards collects real personal data** (UPI ID, name, phone, address). The rules say *pseudonymous accounts only, no real student data*.
> 3. **The README and the slides quote old numbers.** They say 15/15 and 29/29; the latest run is 14/15 and 33/33.

---

1. Requirement-by-requirement check

#	Requirement (from the statement)	Layerflow	Status
1	≥10 challenges, ≥3 difficulty levels, ≥4 concepts	18 challenges · easy / medium / hard · 6 concepts (lists_indexing, loops_range, boundaries, conditionals, accumulators, strings)	✅
2	Each challenge: description, starter code, concept tags, ≥3	Enforced by a unit test on the bank	✅

#	Requirement (from the statement)	Layerflow	Status
	visible + ≥5 hidden tests incl. edge cases		
3	The 4 judge challenges, exactly as specified	`last_k_total(marks, k)` · `count_passes(marks)` · `first_n_squares(n)` · `middle_three(items)`	✅
4	Restricted execution (stronger isolation scores higher)	4 layers: Python policy → Pyodide/WebAssembly → Node permission model + empty env → supervisor (wall clock, memory watchdog, throwaway process)	✅ Strong
5	Every test labelled exactly: passed / wrong output / runtime error / timeout	Fixed classifier · 12/12 judge-style labels exact	✅
6	A memory, output or disk kill = runtime error **that names the limit**	Fixed list of limit names (`memory limit` , `disk limit` , …)	⚠️ The disk flood sometimes comes out as `timeout` (see §3)
7	Hidden tests never revealed (input, expected value, output, exception)	Child process never receives expected values · `hidden[]` is `{test, outcome}` only · database CHECK constraint · 0 leaks in 896 responses and 52 served files	✅
8	Plain-language explanations	Deterministic Tier 1 + optional LLM polish behind the same	✅

#	Requirement (from the statement)	Layerflow	Status
		validator · Line Map · Time-Travel Replay	
9	≥3 progressive hints, never the solution, even when asked	4-rung ladder · validator rejects code · “give me the answer” moves one rung · 4/4 merged programs still fail	✅
10	Mistake tags incl. the 5 required, kept distinct	off_by_one, unhandled_short_input, print_not_return, boundary_comparison, mutates_input (+ wrong_index_direction) · one primary tag per failure	✅
11	Recurring = same tag in ≥2 **different** challenges; shown to the learner	Counts distinct challenge ids · one-off slips listed separately	✅
12	Choose **or generate** the next challenge, with the reason	Next challenge + reason (never empty) · **Bug Factory** generates a buggy program from the top mistake	✅
13	Mastery per concept; an unhinted pass counts more	Beta model: +1.0 unhinted, +0.35 hinted	✅
14	`POST /api/submit`, `POST /api/hint`, `GET /api/learner/{id}`	All three, JSON in the statement’s shape (plus an extra `debug` field)	✅
15	One learner language	Python	✅

#	Requirement (from the statement)	Layerflow	Status
16	Own or attributed content; own hidden tests	All 18 challenges and every hidden test are original	✅
17	Log every AI prompt and response	Hint Ledger (`/ledger`), including chat, mentor and repo calls	✅
18	Pseudonymous accounts only, no real student data	Learners are nicknames ✅ · **Rewards asks for UPI ID, name, phone, address** ❌	❌
19	API reachable by judges from the start of judging	`npm start` + `npm run url` (ngrok), or the laptop	✅ Keep it up and awake
20	Repo + README + one command	`npm install && npm start`	✅
21	Mistake taxonomy on one page	`TAXONOMY.md` + `/taxonomy` + `GET /api/taxonomy`	✅
22	5-minute demo + viva	`docs/viva.md` has the four answers	✅

2. How we score against the judges' tests

Judge test	Points	Our latest result	Risk
Sandbox red-team	15	**14/15** hostile programs contained · 9/9 escape attempts contained · server up	**Fix the disk flood**

Judge test	Points	Our latest result	Risk
Outcome labels	15 (with secrecy)	12/12 exact	Low
Hidden-test secrecy	(with labels)	0 leaks / 896 responses · 0 in 52 files	Low
Hint leakage	10	4/4 merged programs still fail · 0 invalid hints	Low
Recurring mistakes	10	Both planted mistakes found · the one-off slip is not flagged · next challenge targets a recurring mistake	Low
Explanations and hints (human-rated)	15	5 explanations + 4 ladders ready for blind rating	**Medium: the only subjective score**

3. Must fix before the freeze

1. **Disk flood mislabelled.** `open('big','w').write('x' * 10**9)` builds a 1 GB string first. On a busy machine the wall clock fires before the memory watchdog (`apps/runner/src/supervisor.mjs`), and `classify()` puts the wall clock first. Some hidden tests then come out as `timeout` , not `runtime_error · memory limit` .
   Fix: when a hard stop happens, check memory before the wall clock (RSS over budget → `memory`). Or poll the watchdog more often, or lower the budget. Then run `npm run harness` and confirm **15/15**.
2. **Rewards and real data.** Turn off cash-out and goodies shipping for judging, or put them behind a flag. At minimum, keep `/rewards` out of the demo. Collecting UPI IDs, phone numbers and addresses breaks the *"pseudonymous accounts only"* rule.
3. **Update the numbers.** After the re-run, update the README "Self-test results" table and slide 03 of the deck (unit tests are now **33**, not 29).

4. **Navigation.** `Bounty` and `Rewards` sit in the main nav between Practice and Dashboard. Move them to the end or hide them, so the judges see the core flow first.

4. Are the extra features relevant?

Feature	Fit with DI-01	Recommendation
Line Map · Time-Travel Replay	**Direct**: makes explanations specific (15 pts)	Demo it
Bug Factory	**Direct**: the statement's "choose **or generate**"	Demo it
Hint Ledger	**Required** by the AI rule; proves no answer leaks	Show it
`/judges` page (Try it / Break it)	Makes the judges' job easy	Open the slot with it
Hindi / Marathi	Impact for Indian first-semester students	Mention it
Mentoring (Priya + ✨ Summarise)	**Strong fit**: AI-powered learning, same no-code rule	Show 30 s, the wow moment
Ask Priya chat	Fits, but judges *will* ask it for the answer	Keep; show it refusing
Community	Loose fit (peer learning; spoiler blur is on-theme)	Mention only
Repo Analyse	**Off-theme**: reviews GitHub architecture, not beginners' bugs	Don't demo
Bug Bounty	**Off-theme**: security hunting and company programs	Don't demo

Feature	Fit with DI-01	Recommendation
Rewards · cash-out · Dodo payments	**Off-theme + rule conflict** (real personal data)	Hide for judging

Rule of thumb: every minute of the 5-minute demo should map to a scoring row in §2. Extras that don't map dilute the story.

5. What can raise our winning chances

Ranked by the points they protect and how much work each one takes.

1. **Tell the statement's own story.** The judges wrote Rohan's story, so demo it in order:
   - Rohan's starter code: 3/3 visible pass, hidden tests fail, with an explanation that names *the kind of input*.
   - The hint ladder, then "give me the answer", which still moves only one rung.
   - He fixes it himself.
   - The dashboard flags his recurring mistake and the next challenge says why (or the Bug Factory generates one).
   - **Exam day:** he solves a new problem *without hints* and mastery jumps.

   That closes the loop the statement opens ("there is no chatbot in the exam hall").
2. **Win the human-rated 15 points.** Run the T6 sheet as a blind rating inside the team: score each explanation and ladder 0–2 on *correct, specific, not giving it away*. Rewrite anything below 2.
3. **"Exam mode" (small add).** A toggle that hides hints and shows the hinted-vs-unhinted mastery difference. It is the statement's punchline as a feature.
4. **Teacher cohort heatmap (medium add).** Recurring tags across all learners in a class, built from data we already store. It is the strongest *impact* slide for a jury ("61% of the class has boundary_comparison").
5. **Prove isolation live.** On `/judges`, press **Break it** on the fork bomb and the secrets theft, then submit a correct program straight after. Judges remember what they see survive.

6. **Judging-day reliability.** Warm the runner pool before the judges' automated run, keep ngrok and the laptop awake, and watch `GET /api/health`.

6. The 5-minute demo plan

Time	Show	Scores
0:00–0:30	One line: "a coach that never hands over the answer" + `/judges`	—
0:30–2:00	Rohan on `last-k-total`: hidden fails, explanation, Line Map, hint ladder, "give me the answer"	Labels · secrecy · explanations · hint leakage
2:00–2:45	Break it: a hostile program is contained, then the next good submission passes	Red-team
2:45–3:45	`/dashboard?learner=rohan`: recurring vs one-off, mastery, next challenge + reason → Bug Factory	Recurring mistakes · skills
3:45–4:30	Priya's ✨ Summarise replay (30 s) + Ask Priya refusing to give code	Explanations (wow)
4:30–5:00	`/ledger`: every prompt logged, rejected drafts counted	AI rule · trust